

Code Complexity Analysis – Centrality Analysis

Index

- Function 1: getCentralityAnalysisForStation
 - Function 2: calculate_Btw_n_HarmClos
 - Function 3: getALLShortestPaths
 - Function 4: bellmanFord
 - Function 5: buildShortPath
 - Function 6: hasThisStationInPath
 - Function 7: containsPath
 - Function 8: countManyPathsWithoutOriginInStartOrEnd
 - Function 9: calculate_hubScore
-

1. Pseudocode

Function 1: getCentralityAnalysisForStation

```
IF graph is null OR empty OR vertex is null OR invalid
    RETURN null

Initialize ResponseDto with station info
Calculate degree = size of adjacent vertices

IF only one vertex in graph:
    SET harmonicCloseness = 1
    SET betweenness = 1
    SET hubScore = 1
    SET strength = 1
ELSE:
    CALL calculate_Btw_n_HarmClos(g, vOrig, r)
    IF (numVertices - 1) == 0:
        SET strength = 1
    ELSE:
        SET strength = degree / (numVertices - 1)
    SET hubScore = calculate_hubScore(r)

RETURN ResponseDto
```

Function 2: calculate_Btw_n_HarmClos

```

Initialize betweenness = 0
Initialize harmClos = 0
Initialize betweenNessTemp = 0
GET all vertices list
GET all shortest paths using getALLShortestPaths()

FOR each vertex vFrom in graph:
    IF vFrom equals vOrig:
        SKIP

    GET distance from vOrig to vFrom using containsPath()
    IF distance != 0:
        ADD (1 / distance) to harmClos

FOR each path in allPaths:
    IF path does not start or end with vOrig AND path contains vOrig:
        INCREMENT betweenNessTemp

IF allPaths not empty AND paths without origin at endpoints != 0:
    SET betweenness = betweenNessTemp / countManyPathsWithoutOriginInStartOrEnd()

SET harmonicCloseness = harmClos

```

Function 3: getALLShortestPaths

```

GET number of vertices
GET list of all vertices
Initialize empty list of paths

FOR i = 0 to numVertices:
    SET vFrom = vertex at index i
    FOR j = i+1 to numVertices:
        SET vTo = vertex at index j
        Initialize empty path
        IF bellmanFord finds path from vFrom to vTo:
            ADD path to list of paths

RETURN list of paths

```

Function 4: bellmanFord

```

Initialize cost array with POSITIVE_INFINITY
Initialize prev array with -1
SET cost[origin] = 0

FOR i = 0 to numVertices-1:
    SET updated = false
    FOR each edge in graph:
        GET vFrom key and vTo key
        GET edge weight
        IF cost[vFrom] != INFINITY AND cost[vFrom] + weight < cost[vTo]:
            UPDATE cost[vTo] = cost[vFrom] + weight
            UPDATE prev[vTo] = vFrom
            SET updated = true
    IF not updated:
        BREAK (early termination)

IF cost[destination] == INFINITY:
    RETURN false (unreachable)

FOR each edge in graph:
    IF cost[vFrom] + weight < cost[vTo]:
        RETURN false (negative cycle detected)

CALL buildShortPath(g, shortPath, origKey, destKey, prev)
RETURN true

```

Function 5: buildShortPath

```

SET index = vDest

WHILE index != vOrig:
    ADD vertex at index to front of path
    SET index = prev[index]

ADD origin vertex to front of path

```

Function 6: hasThisStationInPath

```
FOR each vertex in path:
    IF vertex equals comparison station:
        RETURN true
RETURN false
```

Function 7: containsPath

```
Initialize total = 0

FOR each path in allPaths:
    IF (path.first == vOrig AND path.last == vFrom) OR
    (path.first == vFrom AND path.last == vOrig):
        FOR j = 0 to path.size - 1:
            GET vertices v1 and vTo
            ADD edge distance to total
        RETURN total

RETURN 0 (path not found)
```

Function 8: countManyPathsWithoutOriginInStartOrEnd

```
SET count = total number of paths

FOR each path in allPaths:
    IF path starts with vOrig OR path ends with vOrig:
        DECREMENT count

RETURN count
```

Function 9: calculate_hubScore

```
RETURN betweenness * 0.35 + harmonicCloseness * 0.35 + strength * 0.3
```

2. Time Complexity Analysis

getCentralityAnalysisForStation

Operation	Description	Complexity	Notes
Input validation	Null/empty checks	1	Constant time
DTO initialization	Create response object	1	Constant time
Calculate degree	adjVertices().size()	1	Direct lookup
Single vertex check	Compare numVertices == 1	1	Constant time
calculate_Btw_n_HarmClos	Centrality calculations	$n^3 \cdot e$	Dominant operation
Calculate strength	Division operation	1	Constant time
calculate_hubScore	Weighted sum	1	Constant time
SUM		$n^3 \cdot e + 6$	
TOTAL		$O(n^3 \cdot e)$	Dominated by centrality calcs

calculate_Btw_n_HarmClos

Operation	Description	Complexity	Notes
Get vertices list	g.vertices()	n	Retrieve all vertices
getALLShortestPaths()	Compute all pairwise paths	$n^2 \cdot e$	$n(n-1)/2$ Bellman-Ford calls
Harmonic closeness loop	Iterate all vertices	n	Outer loop
containsPath() per iteration	Search for path in list	n^2	Scan paths, compute distance
Betweenness loop	Iterate all paths	n^2	Number of paths is $O(n^2)$
hasThisStationInPath()	Check vertex in path	n	Per path check
countManyPaths...()	Count filtered paths	n^2	Iterate all paths
SUM		$n^2 \cdot e + 3n^2 + 3n$	
TOTAL		$O(n^3 \cdot e)$	Dominated by path computation

getALLShortestPaths

Operation	Description	Complexity	Notes
Get numVertices	Graph size query	1	Constant
Get vertices list	Retrieve all vertices	n	Linear
Outer loop	Iterate vertices	n	From i=0 to n
Inner loop	Iterate vertices from i+1	n	Pairwise combinations
bellmanFord() per pair	Shortest path computation	$n \cdot e$	Called $O(n^2)$ times
Add path to list	List operation	1	Amortized constant
SUM		$n \cdot e + 3n + 2$	
TOTAL		$O(n^2 \cdot e)$	n^2 calls $\times O(n \cdot e)$ each

bellmanFord

Operation	Description	Complexity	Notes
Initialize cost array	Fill with INFINITY	n	n vertices
Initialize prev array	Fill with -1	n	n vertices
Set origin cost	cost[orig] = 0	1	Constant
Outer relaxation loop	Up to V-1 iterations	n	Can terminate early
Inner edge loop	Process all edges	n	Per relaxation iteration
Edge relaxation	Update cost and prev	n * e	Per edge
Negative cycle detection	Iterate all edges	e	One final pass
buildShortPath()	Reconstruct path	n	Backtrack through prev array
SUM		n * e + 5n + e + 1	
TOTAL		O(n·e)	Standard Bellman-Ford

Variables:

- n = number of vertices
- e = number of edges

buildShortPath

Operation	Description	Complexity	Notes
While loop	Backtrack through prev	n	At most n vertices
addFirst() operations	Add to front of LinkedList	n	Per iteration
Array access	prev <u>index</u> lookup	n	Per iteration
Final addFirst()	Add origin vertex	1	Constant
SUM		3n + 1	
TOTAL		O(n)	Linear in path length

hasThisStationInPath

Operation	Description	Complexity	Notes
For loop	Iterate path vertices	n	At most n vertices in path
Equality check	vertex.equals()	n	Per vertex
SUM		2n	
TOTAL		O(n)	Linear scan

containsPath

Operation	Description	Complexity	Notes
Outer loop	Iterate all paths	n ²	O(n ²) paths in complete case
Path endpoint check	Compare first/last	n ²	Per path
Inner loop	Calculate path distance	n ²	At most n edges in path

Operation	Description	Complexity	Notes
Edge weight lookup	<code>g.edge().getWeight()</code>	$n^2 * n$	Per edge
SUM		$n^3 + 3n^2$	
TOTAL		$O(n^3)$	$O(n^2)$ paths $\times$ $O(n)$ scan each

Note: In practice, returns on first match, so average case is better.

countManyPathsWithoutOriginInStartOrEnd

Operation	Description	Complexity	Notes
Get <code>allPaths.size()</code>	Total count	1	Constant
For loop	Iterate all paths	n^2	Number of paths
Endpoint comparison	Check first and last	n^2	Per path
SUM		$2n^2 + 1$	
TOTAL		$O(n^2)$	Linear in number of paths

calculate_hubScore

Operation	Description	Complexity	Notes
Weighted calculation	Arithmetic operations	1	Constant
SUM		$n * e + 5n + e + 1$	
TOTAL		$O(1)$	Constant

Overall Complexity

Primary Function: `getCentralityAnalysisForStation`

Overall Time Complexity: $O(n^3 \cdot e)$